Indian Institute of Technology, Madras

DA6400 - Introduction to Reinforcement Learning

Programming Assignment 1 Report

CS24M046 - Shreyas Rathod

May 1, 2025

Option Definitions:

We define four primitive “options” to guide the taxi to each landmark R, G, Y, B. Each option runs until the taxi reaches its target.

```
# ===== Option functions: navigate taxi to each landmark =====

def navigate_to_R(environment, curr_state):
    """
    Macro-action to drive taxi to location 'R'.
    Returns [action, done_flag].
    """
    done_flag = False
    row, col, p_idx, d_idx = environment.unwrapped.decode(curr_state)
    # default: move north
    action_choice = 1

    if row == 0 and col == 0:           # arrived at R
        done_flag = True
    elif col == 0 and row != 0:         # below R in col 0
        action_choice = 1               # north
    elif col == 1 and row <= 2:         # to the right of col 0 but above barrier
        action_choice = 3               # west
    elif row == 2 and col >= 1:         # below the barrier row
        action_choice = 2               # east
    elif row > 0:                       # still above ground row
        action_choice = 0               # south
    elif col == 4:                      # far right column
        action_choice = 3               # west

    return [action_choice, done_flag]


def navigate_to_G(environment, curr_state):
    """
    Macro-action to drive taxi to location 'G'.
    """
    done_flag = False
    row, col, p_idx, d_idx = environment.unwrapped.decode(curr_state)
    # default: move east
    action_choice = 2

    if row == 0 and col == 4:           # arrived at G
        done_flag = True
    elif col == 0 and row <= 2:         # leftmost but above barrier
        action_choice = 1               # north
    elif row == 0 and col < 4:          # top row, not yet at G
        action_choice = 2               # east
    elif row > 0:                       # below top row
        action_choice = 0               # south
    elif col > 4:                       # should not happen, but fallback
        action_choice = 3               # west

    return [action_choice, done_flag]
```

```
def navigate_to_Y(environment, curr_state):
    """
    Macro-action to drive taxi to location 'Y'.
    """
    done_flag = False
    row, col, p_idx, d_idx = environment.unwrapped.decode(curr_state)
    # default: move south
    action_choice = 0

    if row == 4 and col == 0:           # arrived at Y
        done_flag = True
    elif col == 0:                      # first column
        action_choice = 0               # south
    elif col == 1 and row <= 2:         # just right of col 0 above barrier
        action_choice = 3               # west
    elif col == 1 and row > 2:          # just right of col 0 below barrier
        action_choice = 1               # north
    elif col == 2 and row >= 2:         # middle columns below barrier
        action_choice = 3               # west
    elif col == 2 and row < 2:          # middle columns above barrier
        action_choice = 0               # south

    return [action_choice, done_flag]


def navigate_to_B(environment, curr_state):
    """
    Macro-action to drive taxi to location 'B'.
    """
    done_flag = False
    row, col, p_idx, d_idx = environment.unwrapped.decode(curr_state)
    # default: move south
    action_choice = 0

    if row == 4 and col == 3:           # arrived at B
        done_flag = True
    elif col == 0 and row <= 2:         # leftmost above barrier
        action_choice = 2               # east
    elif col == 0 and row > 2:          # leftmost below barrier
        action_choice = 1               # north
    elif col == 1 and row < 2:          # just right of col 0 above barrier
        action_choice = 0               # south
    elif col == 1 and row >= 2:         # just right of col 0 below barrier
        action_choice = 2               # east
    elif col == 2 and row > 2:          # next column below barrier
        action_choice = 1               # north
    elif col == 2 and row <= 2:         # next column above barrier
        action_choice = 2               # east
    elif col == 3:                      # final column before B column
        action_choice = 0               # south
    elif col == 4:                      # far right
        action_choice = 3               # west
```

```
    return [action_choice, done_flag]


# Bundle them into an option lookup
option_actions = {
    6: navigate_to_R,
    7: navigate_to_G,
    8: navigate_to_Y,
    9: navigate_to_B
}
```

SMDP Q Learning with given options :

The following cell implements SMDP Q-Learning with the handcoded options - go_to_R, go_to_G, go_to_Y, go_to_B,

```
# Initialize Q-Table: (States x Actions) === (env.observation_space.n(500) x total
actions(10))
q_values_SMDP = np.zeros((env.observation_space.n, 10))

# Initialize update frequency table
update_freq_SMDP = np.zeros((env.observation_space.n, 10))

# SMDP Q-Learning parameters
gamma = 0.9  # Discount factor
alpha = 0.1  # Learning rate
n_episodes = 5000  # Number of episodes

# Initialize arrays to store rewards and steps per episode
rewards_SMDP = np.zeros(n_episodes)
steps_per_episode_SMDP = []

# Iterate over episodes
for episode in tqdm(range(n_episodes)):
    step_count = 0  # Initialize step count
    state = env.reset()[0]  # Get the initial state
    episode_done = False  # Initialize episode termination flag
    total_rewards = 0  # Initialize total rewards accumulator

    # While episode is not over
    while not episode_done:
        original_state = state  # Store original state before taking action
        step_count += 1  # Increment step count

        # Choose action using epsilon-greedy policy For Learning
        action = egreedy_policy(q_values_SMDP, state, epsilon=0.001)

        # If primitive action (0 to 5)
        if action < 6:
            # Take action and observe next state, reward, and termination flag
            next_state, reward, episode_done, _, _ = env.step(action)
            total_rewards += reward  # Accumulate rewards

            # Usual Q-Learning update rule for state-action pair
            q_values_SMDP[state][action] += alpha * (
                reward + gamma * np.max(q_values_SMDP[next_state]) -
q_values_SMDP[state][action]
            )
            # Increment update frequency for this state-action pair
            update_freq_SMDP[state][action] += 1

            state = next_state  # Move to next state

        reward_bar = 0  # Initialize reward accumulator for option
```

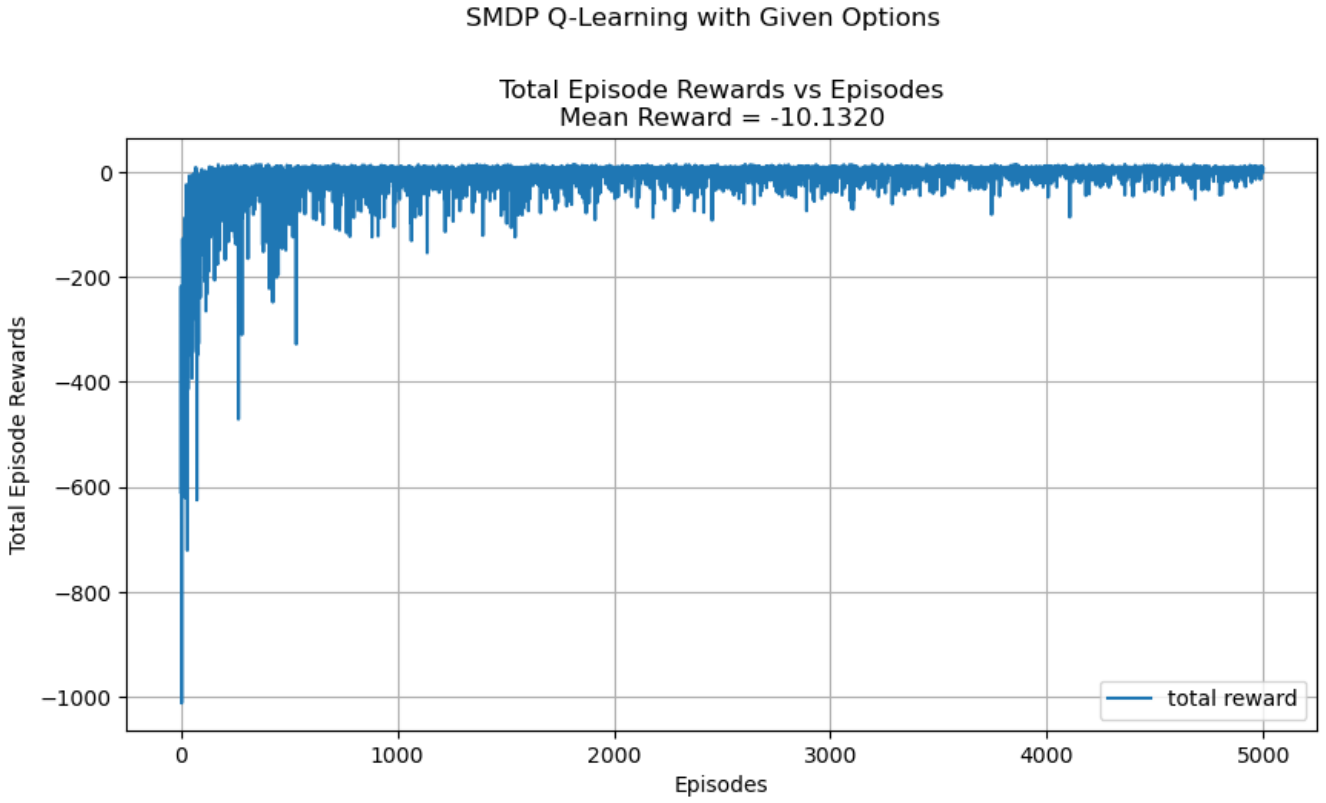
```
        # If option action (6 to 9)
        if action >= 6:
            option_done = False  # Initialize option termination flag
            tau = 0  # Initialize time step for exponentiating gamma

            # Loop until option is done or episode terminates
            while not option_done:
                # Choose action based on current option
                if action == 6:
                    opt_action, option_done = go_to_R(env, state)
                elif action == 7:
                    opt_action, option_done = go_to_G(env, state)
                elif action == 8:
                    opt_action, option_done = go_to_Y(env, state)
                elif action == 9:
                    opt_action, option_done = go_to_B(env, state)

                # Take action and observe next state, reward, and termination flag
                next_state, reward, episode_done, _, _ = env.step(opt_action)
                total_rewards += reward  # Accumulate total rewards

                # Update the discounted reward
                reward_bar += (gamma**tau) * reward  # Update reward_bar with discounted
reward
                tau += 1  # Increment time step

                # Update Q Value when Option is terminated
                if option_done or episode_done:
                    # Update Q-values using SMDP Q-learning update rule
                    q_values_SMDP[original_state][action] += alpha * (
                        reward_bar + (gamma**tau) * np.max(q_values_SMDP[state]) -
q_values_SMDP[original_state][action]
                    )
                    # Increment update frequency for this state-action pair
                    update_freq_SMDP[original_state][action] += 1

                state = next_state  # Move to next state

    rewards_SMDP[episode] = total_rewards  # Store total rewards for the episode
    steps_per_episode_SMDP.append(step_count)  # Store number of steps taken in the
episode
```

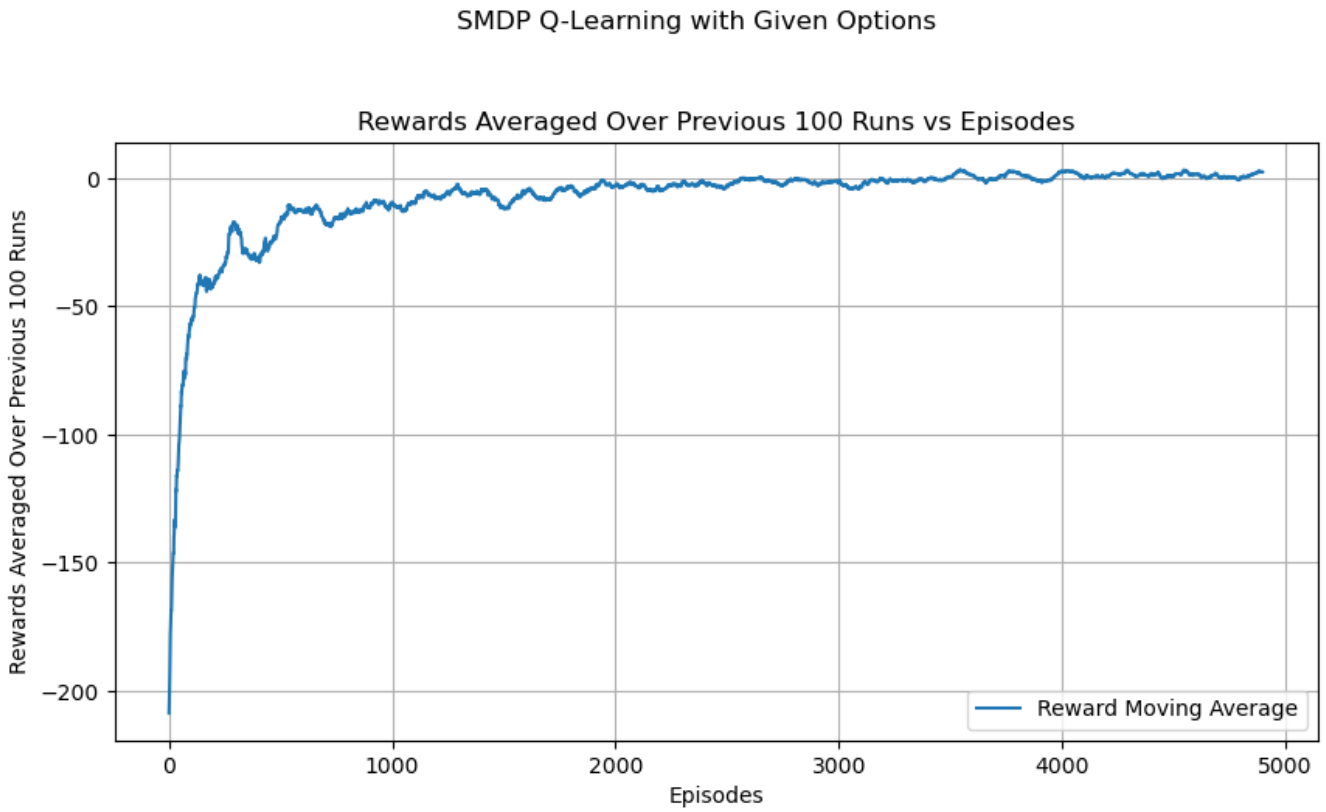
Observations :

HYPER-PARAMTERS USED :

- Alpha (Learning rate) = 0.1
- Gamma (Discount factor) = 0.9
- Epsilon (Epsilon Greedy) =0.001
- No of episodes = 5000

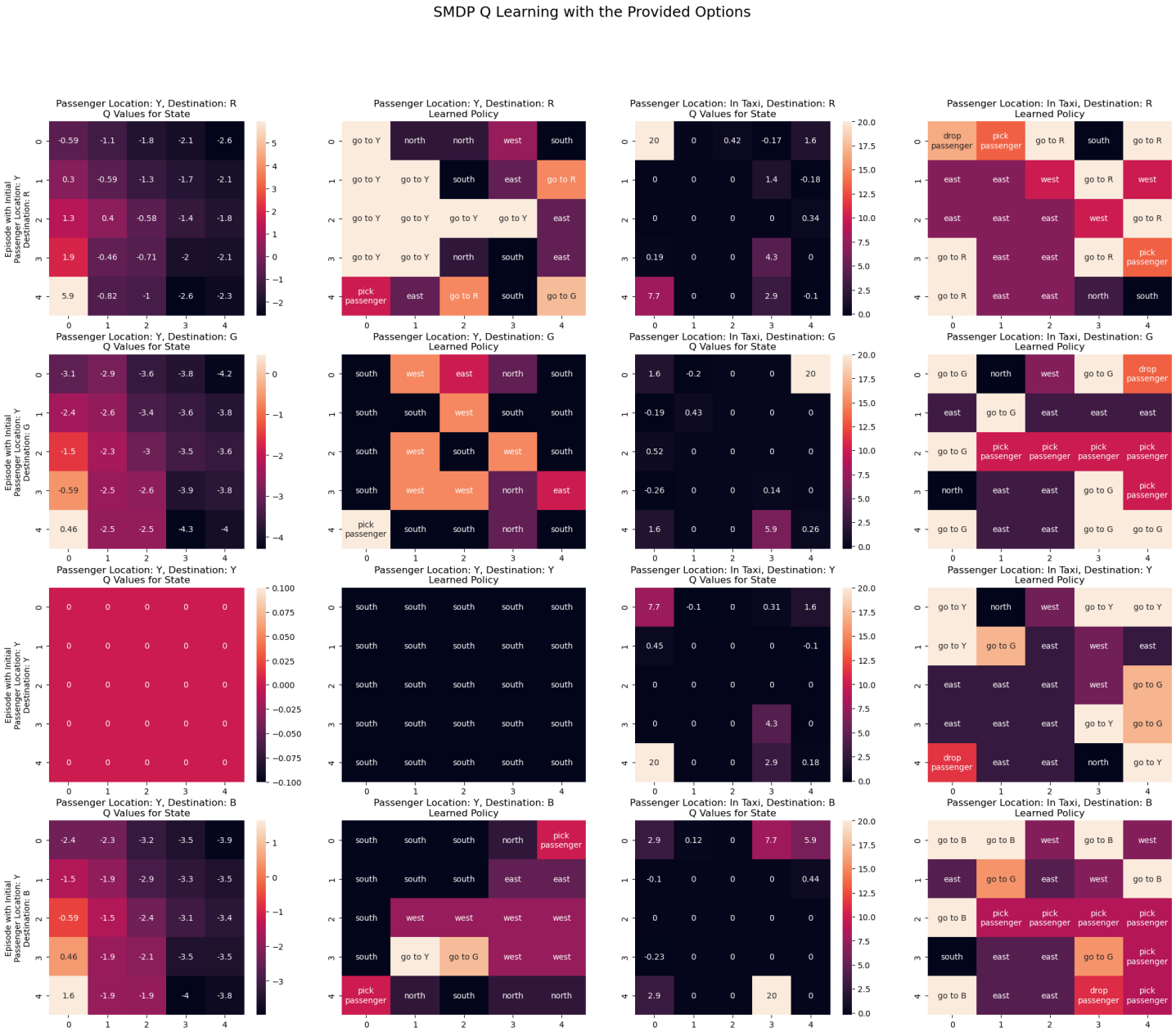
Reward plot for SMDP with given option :





- The reward progression demonstrates gradual improvement, eventually stabilizing at a mean reward of **-10.13**. This convergence occurs at a slower rate compared to Intra-Option Q-Learning.

Q-VALUES PLOT FOR SMDP (for passenger location =2) :



Subjective Markov Decision Process (SMDP) Q-learning builds upon traditional Q-learning by incorporating "options." This allows the learning agent to select not just basic actions, but also more complex, temporally extended actions known as options. Essentially, SMDP Q-learning learns a Q-value for every possible combination of an option and an action.

Options provide the agent with the ability to operate at different speeds or timeframes. They represent sequences of fundamental actions that the agent can choose to execute. Each option is defined by a specific policy (how to act within the option), a condition for when the option should end, and a set of conditions for when it can be initiated.

By using options, SMDP Q-learning introduces a hierarchical structure to the learning process. These options function at different levels of abstraction, enabling the agent to make decisions across various temporal scales.

Observations from the provided plot indicate a tendency for the agent to select options more frequently than individual, primitive actions.

Analyzing the Q-value plots for the "person in Taxi" scenario reveals the following:

- **To reach location R, the chosen option correctly aimed to drop off the passenger at the intended spot, even though the immediate primitive actions taken were not always the most efficient.**
- **Similarly, the option for reaching location G focused on the correct drop-off point, but only a limited number of the underlying primitive actions were optimal.**
- **When trying to reach location Y, the agent sometimes chose to move east, even when going west appeared to be a better immediate choice due to obstacles.** This suggests the option might be considering a longer-term strategy.
- **To reach location B, the agent sometimes selected option G and then option B.** This illustrates how options can be chained together.
- **Across all considered options, the final drop-off location was consistently correct.**

In essence, SMDP Q-learning with options allows the agent to learn and execute more sophisticated behaviors by considering sequences of actions as single units, potentially leading to efficient goal achievement even if individual steps aren't always immediately optimal.

Intra Option Q Learning with given options :

Intra option is Off Policy Algorithm so we have used Epsilon Greedy to Learn and For the update in Q value will be depends on the option termination thus it has Different Behavior policy and Different Evaluation Policy.

```
q_values_IOQL = np.zeros((500, 10))
update_freq_IOQL = np.zeros((500, 10))

gamma = 0.9
alpha = 0.1
rewards_IOQL = []
steps_IOQL = []
n_eps = 5000

for episode in tqdm(range(n_eps)):
    total_rewards = 0
    step = 0
    state = env.reset()[0]
    done = False

    while not done:
        step += 1
        action = egreedy_policy(q_values_IOQL, state, epsilon=0.001)

        if action < 6:
            next_state, reward, done, _, _ = env.step(action)
            total_rewards += reward

            q_values_IOQL[state][action] += alpha * (reward + gamma *
np.max(q_values_IOQL[next_state]) - q_values_IOQL[state][action])
            update_freq_IOQL[state][action] += 1
            state = next_state

        if action >= 6:
            optdone = False
            while not optdone and not done:
                optact, optdone = option_dict[action](env, state)
                next_state, reward, done, _, _ = env.step(optact)
                total_rewards += reward

                q_values_IOQL[state][optact] += alpha * (reward + gamma *
np.max(q_values_IOQL[next_state]) - q_values_IOQL[state][optact])
                update_freq_IOQL[state][optact] += 1

                q_values_IOQL[state][action] += alpha * (reward + gamma * ((1 - optdone)
* q_values_IOQL[next_state][action] + optdone * np.max(q_values_IOQL[next_state])) -
q_values_IOQL[state][action])
                update_freq_IOQL[state][action] += 1

                taxi_row, taxi_col, passenger_index, destination_index =
env.unwrapped.decode(state)
                if (action == 6 or action == 8) and taxi_col > 0:
                    for opt in range(6, 10):
                        if opt != action:
```

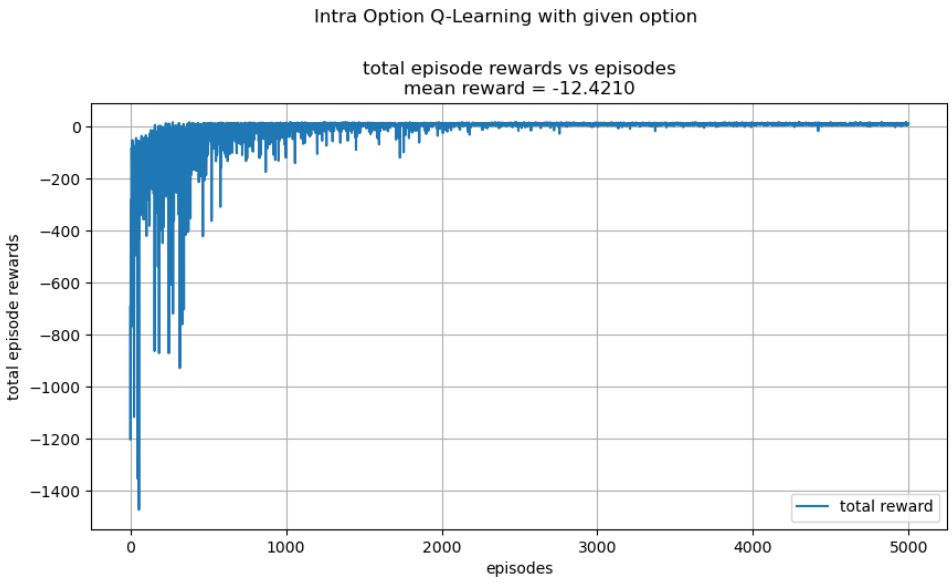
```
                            optaction_other, optdone_other = option_dict[opt](env, state)
                            if optact == optaction_other:
                                q_values_IOQL[state][opt] += alpha * (reward + gamma *
((1 - optdone_other) * q_values_IOQL[next_state][opt] + optdone_other *
np.max(q_values_IOQL[next_state])) - q_values_IOQL[state][opt])
                                update_freq_IOQL[state][opt] += 1

                if (action == 7 or action == 9) and taxi_col < 3:
                    for opt in range(6, 10):
                        if opt != action:
                            optaction_other, optdone_other = option_dict[opt](env, state)
                            if optact == optaction_other:
                                q_values_IOQL[state][opt] += alpha * (reward + gamma *
((1 - optdone_other) * q_values_IOQL[next_state][opt] + optdone_other *
np.max(q_values_IOQL[next_state])) - q_values_IOQL[state][opt])
                                update_freq_IOQL[state][opt] += 1

                state = next_state

    rewards_IOQL.append(total_rewards)
    steps_IOQL.append(step)
```

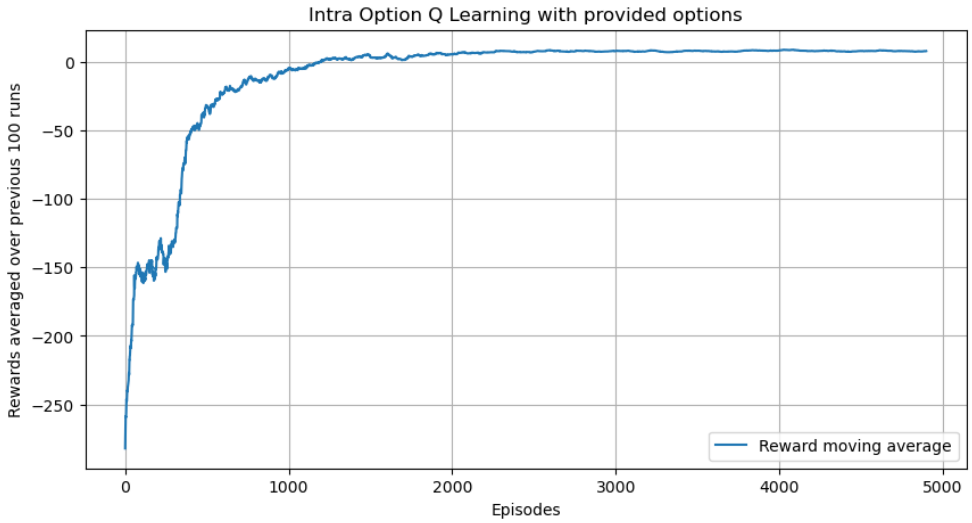
OBSERVATIONS :

HYPERPARAMTERS USED :

- Alpha (Learning rate) = 0.1
- Gamma (Discount factor) = 0.9
- Epsilon (Epsilon Greedy) =0.001
- No of episodes = 5000

REWARD PLOT FOR INTRA OPTION :





- The reward plot shows increased fluctuation and an average reward of **-12.42**. However, it achieves convergence at a faster pace than traditional SMDP Q-learning, unlike the smoother reward progression seen in one-step SMDP.

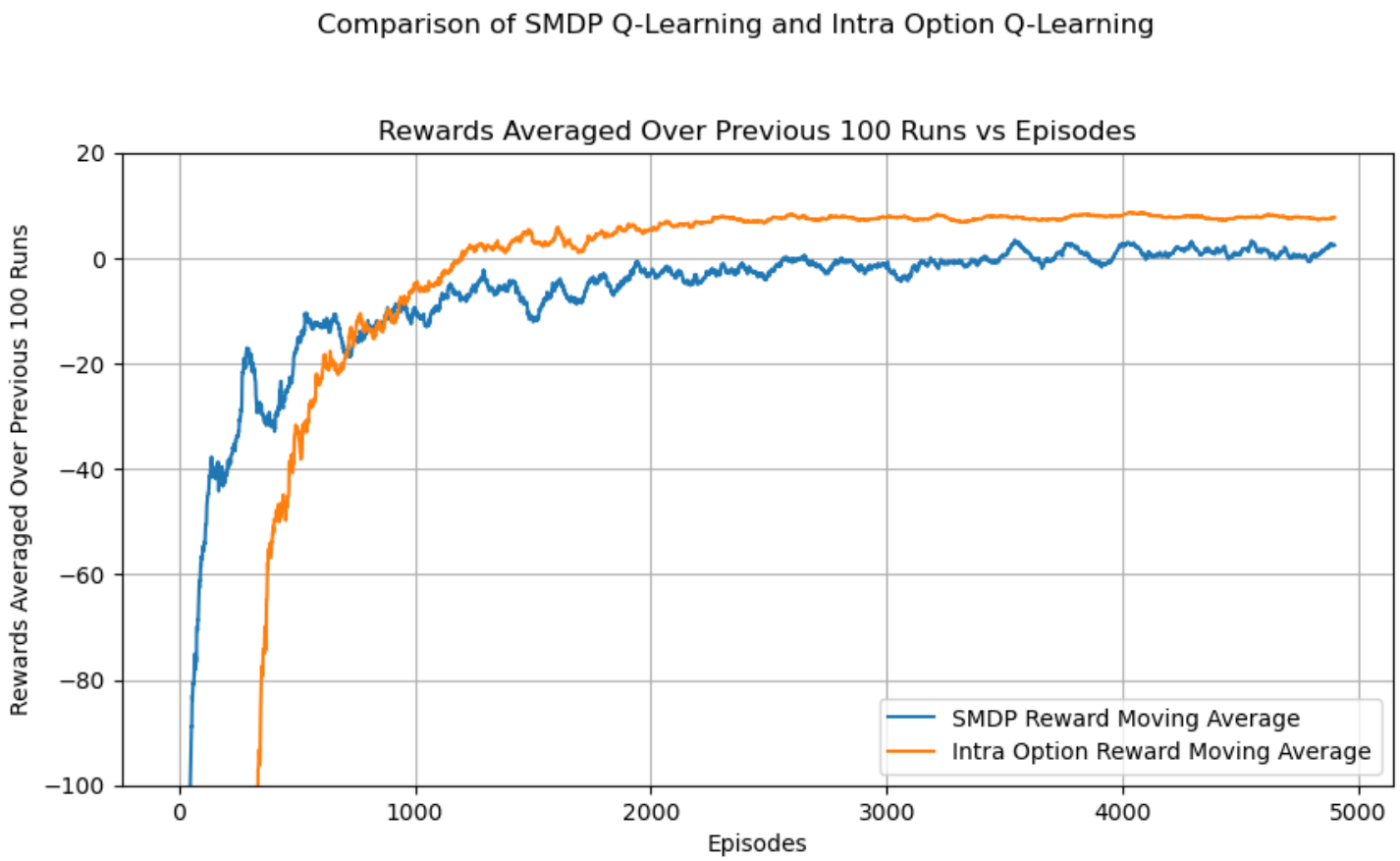
Q-VALUES PLOTS FOR INTRA OPTION :

- **Intra-option Q-learning empowers agents to think in terms of temporally extended actions, or "options," which represent more sophisticated behaviors.** This ability to abstract actions can significantly streamline decision-making within intricate environments.
- **By learning policies for these options, agents can explore the environment more effectively by taking longer, more informative steps.** This can lead to quicker learning and improved convergence, particularly in scenarios characterized by infrequent rewards or long-term goals.
- **Intra-option Q-learning offers considerable flexibility in how options are defined, including their termination conditions and the situations in which they can be initiated.**
- **A key aspect of Intra-option Q-learning is the added layer of decision-making required at each step: the agent must select the appropriate option to execute.** If this option selection process is not carefully designed, it could result in less than optimal performance or inefficient exploration.
- **Our observations from the plot indicate a tendency for the agent to favor the selection of options over individual, primitive actions.**

Analyzing the Q-value plots for the "person in Taxi" environment reveals the following insights:

- **For the option aiming to reach location R, the agent sometimes attempts to go towards G (another location at the top of the taxi environment) from certain bottom states. However, the majority of actions taken correctly lead towards R.**
- **Regarding the option for reaching location G, only two states are associated with incorrect actions (south-east), while the rest are correctly mapped.**
- **For the option targeting location Y, most actions correctly guide the agent towards Y, with only a few actions leading towards B.**
- **Concerning the option to reach location B, almost all choices are correct, with the exception of a single instance where the agent chooses option G.**
- **Across all the examined options, the final drop-off location was consistently accurate.**

Comparison between SMDP Q Learning and Intra Option Q Learning : **(with given options)**



- Compared to intra-option Q-learning, 1-step SMDP Q-learning offers a simpler approach to implementation. Its focus on learning Q-values for individual actions makes it more straightforward to grasp and put into practice.
- In scenarios with short timeframes or where using options doesn't offer a substantial benefit, 1-step SMDP Q-learning can be more computationally efficient. This is because it avoids the added complexity of learning policies for options.
- Conversely, intra-option Q-learning allows the agent to reason at a higher level by abstracting over sequences of actions. In grid-world environments, this capability can lead to more effective navigation towards distant objectives.
- By learning how to execute options, the agent in intra-option Q-learning can explore the environment more efficiently, particularly when rewards are infrequent or goals are far in the future. This can accelerate the learning process and improve how quickly the agent converges to a good solution.
- The provided plot indicates that Intra-option Q-learning achieves convergence more rapidly than its one-step counterpart, suggesting superior performance in this specific context.
- Interestingly, 1-step SMDP Q-learning often exhibits faster initial learning, especially in environments with short-term tasks or where options don't provide a significant advantage. This initial speed is confirmed by our plot.

SMDP Q Learning with Alternate set of options

(1) Go to left Center

(2) Go to right Center

```
# Define Q-Table and Update Frequency data structures
q_values_new_op_SMDP = np.zeros((500, 8))
update_frequency_new_op_SMDP = np.zeros((500, 8))

# Define parameters
gamma = 0.9  # Discount factor
alpha = 0.1  # Learning rate
rewards_new_op_SMDP = []  # List to store rewards for each episode
steps_new_op_SMDP = []  # List to store number of steps for each episode
n_eps = 5000  # Number of episodes

# Iterate over episodes
for episode in tqdm(range(n_eps)):
    state = env.reset()[0]  # Get the initial state
    done = False  # Set episode termination flag to False
    total_rewards = 0  # Initialize total rewards for the episode
    step = 0  # Initialize step count for the episode

    # While episode is not over
    while not done:
        step += 1  # Increment step count
        # Choose action using epsilon-greedy policy
        action = egreedy_policy(q_values_new_op_SMDP, state, epsilon=0.001)

        # If the chosen action is a primitive action
        if action < 6:
            # Perform regular Q-Learning update for state-action pair
            next_state, reward, done, _, _ = env.step(action)
            # Update Q-values using Q-Learning update rule
            q_values_new_op_SMDP[state][action] += alpha * (reward + gamma *
np.max(q_values_new_op_SMDP[next_state]) - q_values_new_op_SMDP[state][action])
            # Update frequency
            update_frequency_new_op_SMDP[state][action] += 1
            # Update total rewards
            total_rewards += reward
            # Update current state
            state = next_state

        # If the chosen action is an option
        reward_bar = 0  # Initialize cumulative reward within an option

        if action == 6:  # Option: go to left center
            optdone = False  # Initialize option termination flag
            org_state = state  # Store original state
            tau = 0  # Initialize timestep within the option
            while not optdone:  # Continue until option is complete
                optact, optdone = go_to_left_center(env, state)  # Execute option
```

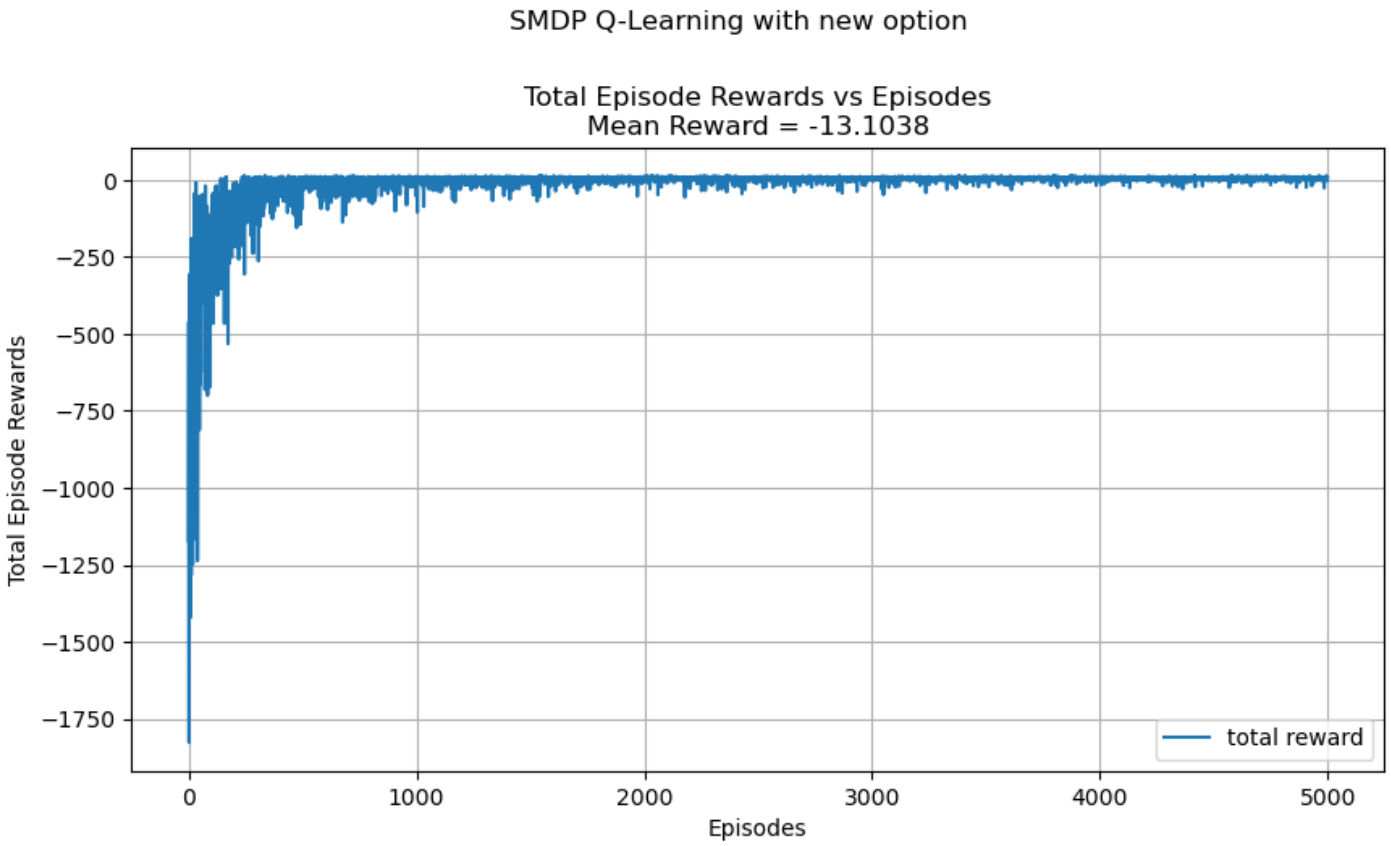
```
                next_state, reward, done, _, _ = env.step(optact)  # Get next state and
reward

                total_rewards += reward  # Update total rewards

                reward_bar += (gamma**tau) * reward   # Update cumulative reward
                tau += 1  # Increment timestep within the option

                # Update Q-values using SMDP Q-Learning update rule when option is
complete
                if optdone or done:
                    q_values_new_op_SMDP[org_state][action] += alpha * (reward_bar +
(gamma ** tau) * np.max(q_values_new_op_SMDP[next_state]) -
q_values_new_op_SMDP[org_state][action])
                    update_frequency_new_op_SMDP[org_state][action] += 1

                state = next_state  # Update current state

        if action == 7:  # Option: go to right center
            optdone = False  # Initialize option termination flag
            org_state = state  # Store original state
            tau = 0  # Initialize timestep within the option
            while not optdone:  # Continue until option is complete
                optact, optdone = go_to_right_center(env, state)  # Execute option
                next_state, reward, done, _, _ = env.step(optact)  # Get next state and
reward

                total_rewards += reward  # Update total rewards

                reward_bar += (gamma**tau) * reward   # Update cumulative reward
                tau += 1  # Increment timestep within the option

                # Update Q-values using SMDP Q-Learning update rule when option is
complete
                if optdone or done:
                    q_values_new_op_SMDP[org_state][action] += alpha * (reward_bar +
(gamma ** tau) * np.max(q_values_new_op_SMDP[next_state]) -
q_values_new_op_SMDP[org_state][action])
                    update_frequency_new_op_SMDP[org_state][action] += 1

                state = next_state  # Update current state

    rewards_new_op_SMDP.append(total_rewards)  # Append total rewards for the episode
    steps_new_op_SMDP.append(step)  # Append step count for the episode
```

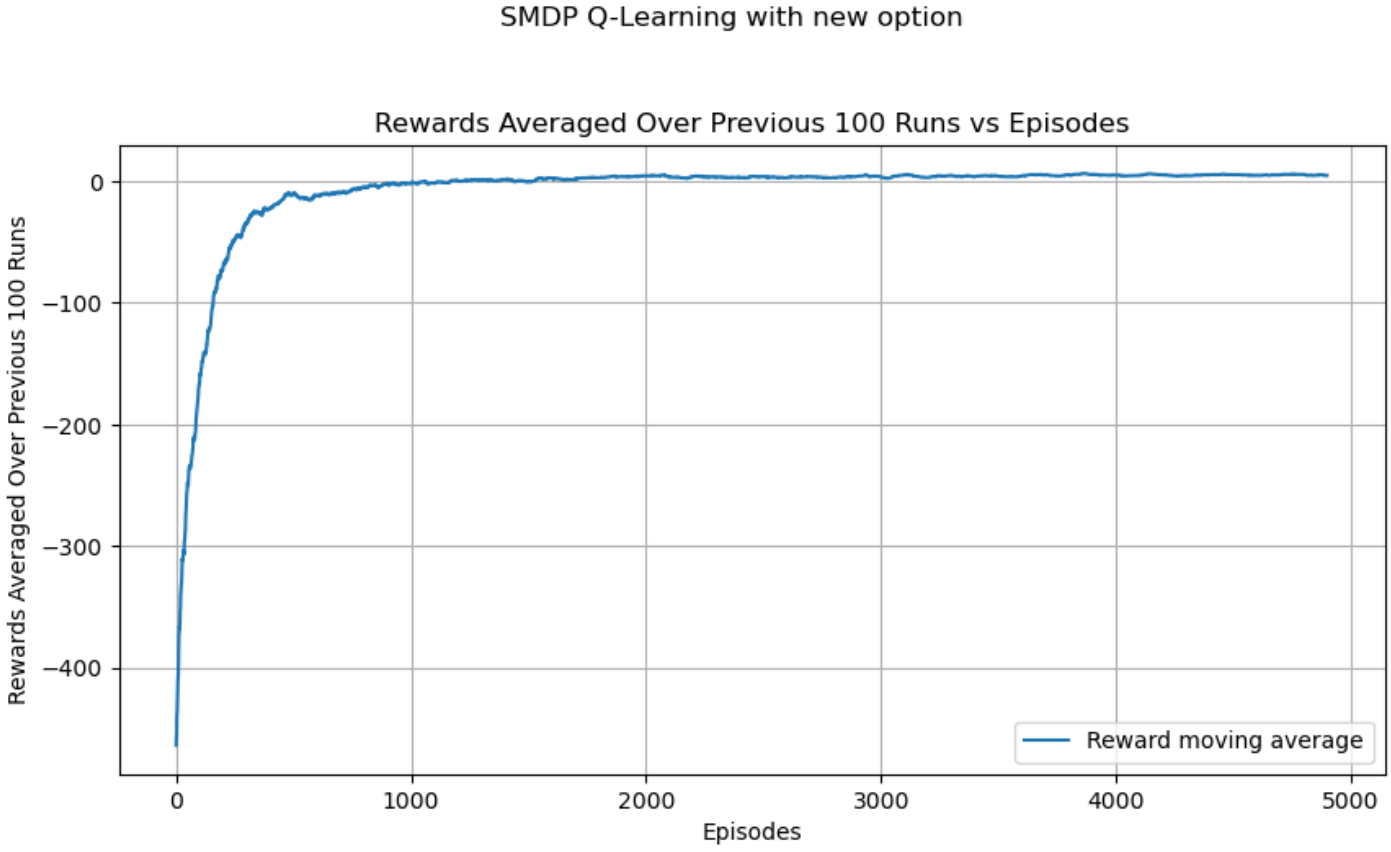
OBSERVATIONS :

HYPER-PARAMTERS USED:

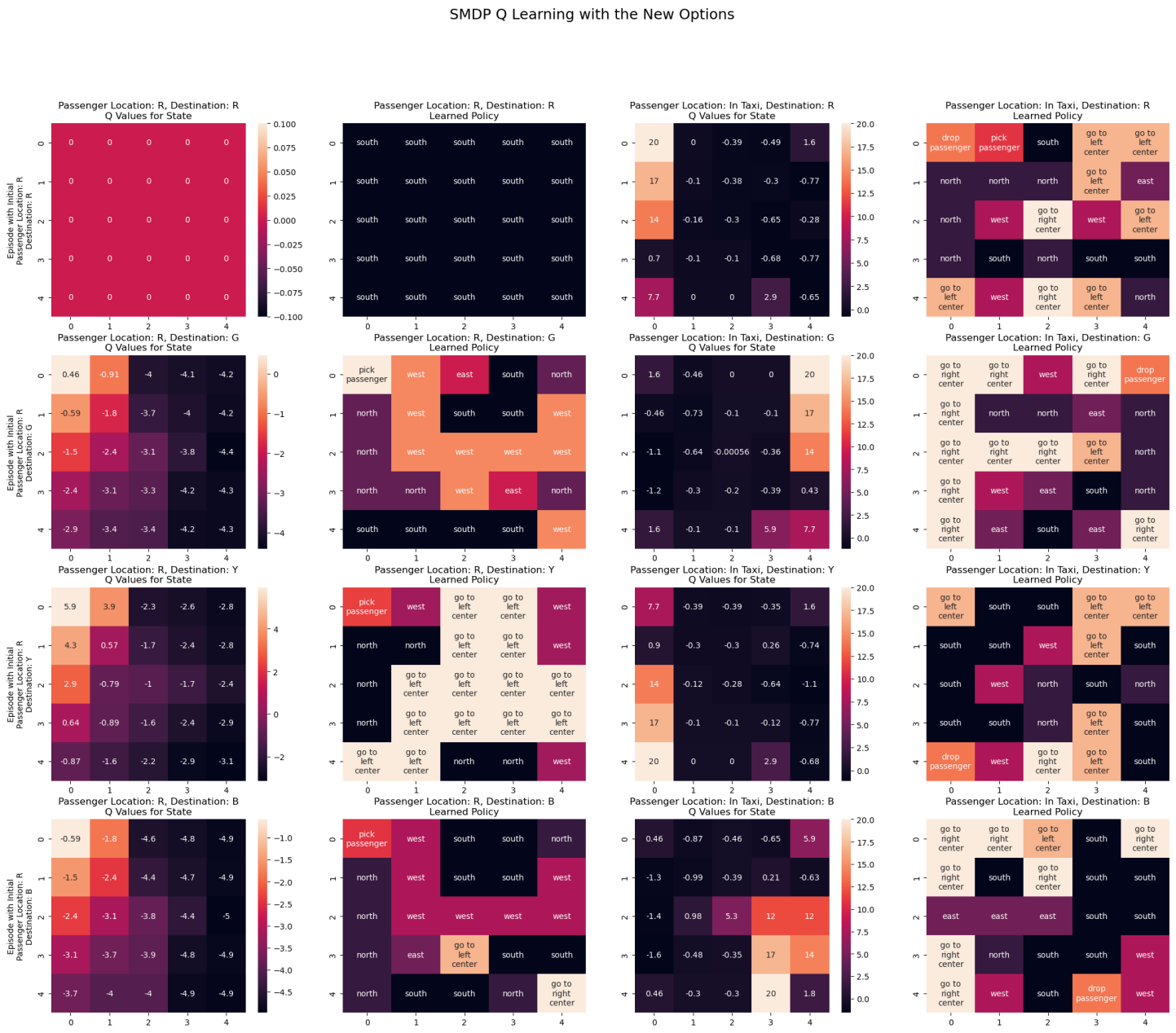
- Alpha (Learning rate) = 0.1
- Gamma (Discount factor) = 0.9
- Epsilon (Epsilon Greedy) =0.001
- No of episodes = 5000

REWARD PLOT FOR SMDP WITH ALTERNATE SET OF OPTIONS :





The rewards plot illustrates a clear improvement over successive episodes, eventually plateauing with an average reward of **-13.10**. However, the convergence to this level is slower when compared to the intra-option approach.



- We experimented with two new options: "go to left center," which targets the center cell of the leftmost column, and "go to right center," which aims for the center cell of the rightmost column.
- As indicated in the plot above, we observed the agent selecting options more frequently than primitive actions.
- SMDP Q-learning, when incorporating options, introduces a hierarchical decision-making framework. Options operate at different levels of abstraction, enabling the agent to make choices across various time scales.
- We also observed similar behavior when using an alternative set of options, suggesting consistency with the patterns seen with the given options.

Analyzing the Q-value plots for the "person in Taxi" scenario reveals the following:

- **For the option to reach R, the agent primarily attempts to drop off the passenger at the correct location.** Only in one state does it choose to go to the right center, likely due to an obstruction, but otherwise, the learning is functioning well.
- **Regarding the option to reach G, the agent largely aims for the correct drop-off location.** Only a small number of states result in the selection of incorrect actions.
- **For the option to reach Y, the agent mostly selects the optimal action.** However, due to an obstruction, it occasionally chooses a different action.
- **Concerning the option to reach B, the majority of actions are correct.** Some incorrect actions are taken, likely as a consequence of an obstruction.
- **Across all options, the final drop-off locations are correctly identified.**

Intra Option Q Learning with new options

(1) Go to left Center

(2) Go to right Center

```
# Q-Table: (States x Actions) === (env.ns(500) x total actions(8))
q_values_new_op_IOQL = np.zeros((500, 8))

# Update Frequency Data structure
update_frequency_new_op_IOQL = np.zeros((500, 8))

# Add parameters
gamma = 0.9  # Discount factor
alpha = 0.1  # Learning rate

rewards_new_op_IOQL = []  # Store rewards for each episode
steps_new_op_IOQL = []  # Store steps for each episode

n_eps = 5000  # Number of episodes

# Iterate over episodes
for episode in tqdm(range(n_eps)):
    total_rewards = 0  # Initialize total rewards for the episode
    state = env.reset()[0]  # Get the initial state
    done = False  # Initialize done flag for episode termination
    step = 0  # Initialize step counter for the episode

    # While episode is not over
    while not done:
        step += 1  # Increment step counter

        # Choose action
        action = egreedy_policy(q_values_new_op_IOQL, state, epsilon=0.001)

        # Checking if primitive action
        if action < 6:
            # Perform regular Q-Learning update for state-action pair
            next_state, reward, done, _, _ = env.step(action)
            total_rewards += reward

            # Update Q values and update frequency for primitive action
            q_values_new_op_IOQL[state][action] += alpha * (reward + gamma *
np.max(q_values_new_op_IOQL[next_state]) - q_values_new_op_IOQL[state][action])
            update_frequency_new_op_IOQL[state][action] += 1

            state = next_state  # Update current state

        # Checking if action chosen is an option
        if action == 6:  # Option: go to left center
            optdone = False
            while not optdone and not done:
                optact, optdone = go_to_left_center(env, state)  # Take option action
```

```
                next_state, reward, done, _, _ = env.step(optact)
                total_rewards += reward

                # Update Q values and update frequency for sub action of current option
                q_values_new_op_IOQL[state][optact] += alpha * (reward + gamma *
np.max(q_values_new_op_IOQL[next_state]) - q_values_new_op_IOQL[state][optact])
                update_frequency_new_op_IOQL[state][optact] += 1

                # Update Q value and update frequency for the other option which follows
same policy as current option
                taxi_row, taxi_col, passenger_index, destination_index =
env.unwrapped.decode(state)
                if optact == 7 and (taxi_row < 2 or taxi_row > 2):
                    q_values_new_op_IOQL[state][7] += alpha * (reward + gamma * (optact *
np.max(q_values_new_op_IOQL[next_state]) + (1 - optact) *
q_values_new_op_IOQL[next_state][7]) - q_values_new_op_IOQL[state][7])
                    update_frequency_new_op_IOQL[state][7] += 1

                # Update Q values and update frequency for current option
                q_values_new_op_IOQL[state][action] += alpha * (reward + gamma *
((optdone) * np.max(q_values_new_op_IOQL[next_state]) + (1 - optdone) *
(q_values_new_op_IOQL[next_state][action])) - q_values_new_op_IOQL[state][action])
                update_frequency_new_op_IOQL[state][action] += 1

                state = next_state

        if action == 7:  # Option: go to right center
            optdone = False
            while not optdone and not done:
                optact, optdone = go_to_right_center(env, state)  # Take option action
                next_state, reward, done, _, _ = env.step(optact)
                total_rewards += reward

                # Update Q values and update frequency for sub action of current option
                q_values_new_op_IOQL[state][optact] += alpha * (reward + gamma *
np.max(q_values_new_op_IOQL[next_state]) - q_values_new_op_IOQL[state][optact])
                update_frequency_new_op_IOQL[state][optact] += 1

                # Update Q value and update frequency for the other option which follows
same policy as current option
                taxi_row, taxi_col, passenger_index, destination_index =
env.unwrapped.decode(state)
                if optact == 6 and (taxi_row < 2 or taxi_row > 2):
                    q_values_new_op_IOQL[state][6] += alpha * (reward + gamma * (optact *
np.max(q_values_new_op_IOQL[next_state]) + (1 - optact) *
q_values_new_op_IOQL[next_state][6]) - q_values_new_op_IOQL[state][6])
                    update_frequency_new_op_IOQL[state][6] += 1

                # Update Q values and update frequency for current option
                q_values_new_op_IOQL[state][action] += alpha * (reward + gamma *
((optdone) * np.max(q_values_new_op_IOQL[next_state]) + (1 - optdone) *
(q_values_new_op_IOQL[next_state][action])) - q_values_new_op_IOQL[state][action])
                update_frequency_new_op_IOQL[state][action] += 1
```

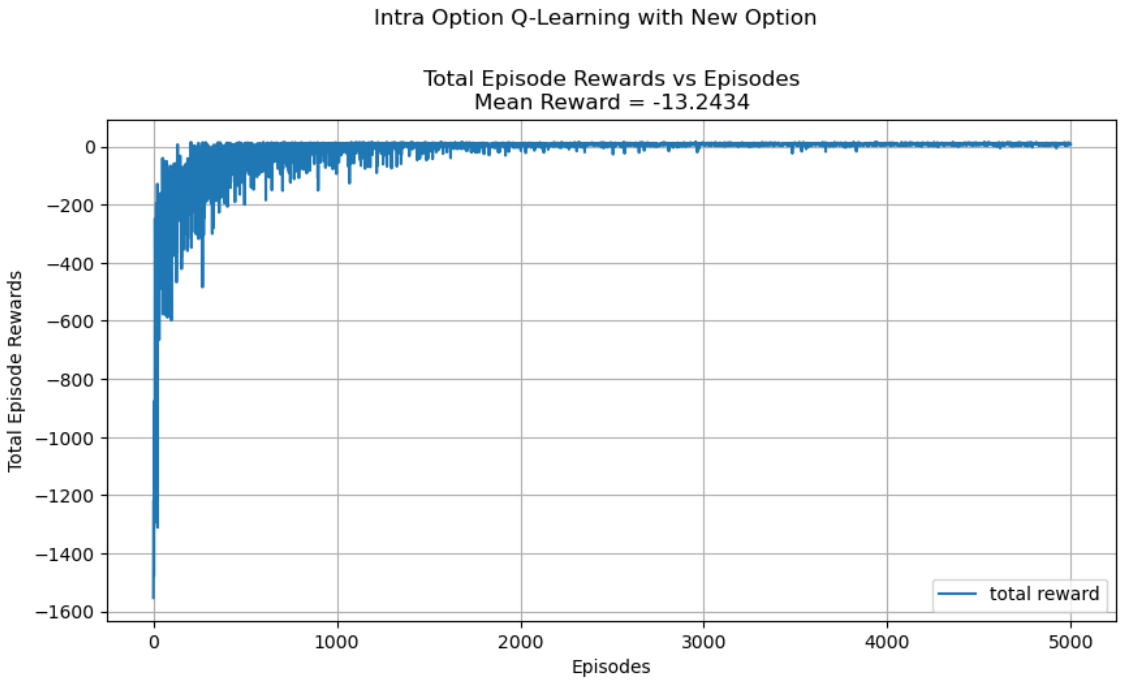
```
            state = next_state

    rewards_new_op_IOQL.append(total_rewards)  # Append total rewards for the episode
    steps_new_op_IOQL.append(step)  # Append steps for the episode
```

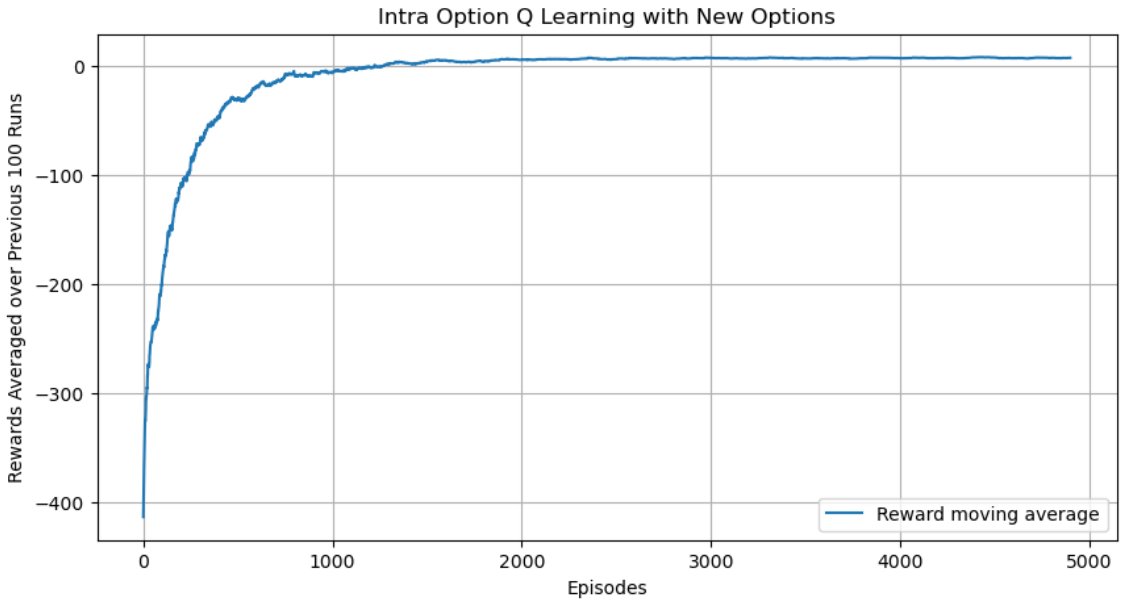
OBSERVATIONS :

HYPERPARAMTERS USED:

- Alpha (Learning rate) = 0.1
- Gamma (Discount factor) = 0.9
- Epsilon (Epsilon Greedy) =0.001
- No of episodes = 5000

REWARD PLOT FOR INTRA OPTION WITH ALTERNATE SET OF OPTIONS:





- We can see from the reward plot that the rewards fluctuate substantially, yielding a mean of **-13.24**. However, it reaches convergence more rapidly than traditional SMDP Q-learning. The one-step SMDP shows less of this fluctuation.

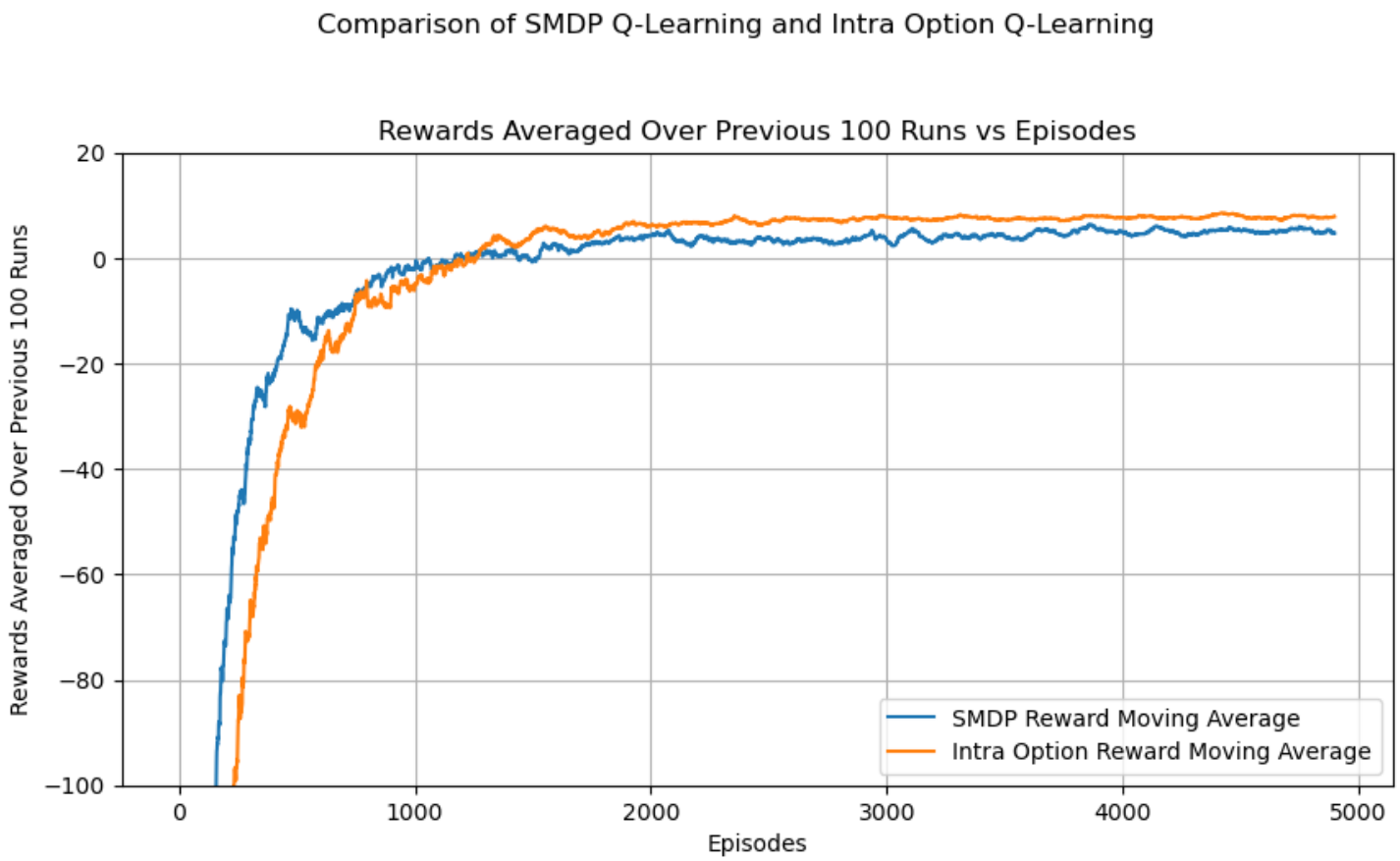
- **Learning policies for options enables the agent to explore the state space more efficiently by taking longer, more informative steps.** This can lead to quicker learning and improved convergence, particularly in tasks where rewards are infrequent or the time horizons are long.
- **Intra-option Q-learning offers considerable flexibility in defining the characteristics of options, such as their termination conditions and the states in which they can be initiated.**
- **We introduced two new options: "go to left center," which directs the agent to the center cell of the leftmost column, and "go to right center," which guides it to the center cell of the rightmost column.**

- **Contrary to previous observations, the plot above indicates that the agent is now choosing more primitive actions than options.**
- **We also noted similar behavioral patterns when an alternative set of options, similar to the given ones, was employed.**

Analyzing the Q-value plots for the "person in Taxi" scenario reveals the following insights:

- **For the option aiming to reach location R, the agent primarily tries to drop off the passenger correctly.** Only in two states does it choose a wrong action, but this is also influenced by an obstruction; otherwise, the learning is effective.
- **Regarding the option for reaching location G, the agent generally aims for the correct drop-off location.** Only a few states lead to the selection of "west" as a wrong action, and a single state chooses "south" incorrectly, with all other actions being appropriate.
- **For the option targeting location Y, the agent mostly selects the optimal action.** However, it sometimes chooses to go to the right option.
- **Concerning the option to reach location B, the majority of actions are correct.** Some incorrect actions are taken, likely due to an obstruction.
- **Across all options, the final drop-off locations are correctly identified.**

Comparison between SMDP Q Learning and Intra Option Q Learning : (With New Options)



- Compared to intra-option Q-learning, 1-step SMDP Q-learning is easier to implement due to its focus on learning Q-values for individual actions, making it more readily understandable and applicable.
- In environments where tasks have short durations or where the use of options doesn't provide a significant advantage, 1-step SMDP Q-learning can be more computationally efficient as it avoids the overhead of learning option policies.
- Conversely, intra-option Q-learning enables the agent to reason at a higher level of abstraction by considering temporally extended actions. This can lead to more efficient navigation toward distant goals, particularly in grid-world environments.
- By learning policies for options, intra-option Q-learning allows the agent to explore the state space more effectively, especially in scenarios with infrequent rewards or long-term objectives, potentially accelerating learning and improving convergence.
- The plot above clearly demonstrates that Intra-option Q-learning performs better than one-step SMDP by achieving convergence at a faster rate.
- It's worth noting that 1-step SMDP Q-learning tends to exhibit faster initial learning compared to intra-option Q-learning, particularly in environments with short task horizons or where options offer limited benefits. Our plot confirms this characteristic.
- Consequently, with the new alternative set of options, the agent behaves similarly to its performance with the original options, and Intra-option learning clearly outperforms SMDP.

GitHub : https://github.com/Shreyas-Rathod/DA6400_JanMay2025_PA3.git